

A PROJECT REPORT

on

“A Comparative Performance Analysis of AES and ASCON Cryptographic Suites on HPC Architectures”

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

**BACHELOR’S DEGREE IN COMPUTER SCIENCE &
ENGINEERING**

BY

Name	Roll Number
Ayush Maity	2205545
Arya Rustagi	22053927
Yash Saxena	22054004
Ujaala Rajgariha	22053996
Shreyas Chakraborty	2205593
Kinshuk Bhattacharya	22053962

**SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA - 751024**

November 2026

CERTIFICATE



This is to certify that the project entitled
**“Comparative Performance Analysis of AES and ASCON
Cryptographic Suites on HPC Architectures”**

submitted by

Ayush Maity (2205545), Arya Rustagi (22053927), Shreyas Chakraborty
(2205593), Ujaala Rajgariha (22053996), Yash Saxena (22054004), Kinshuk
Bhattacharya (2205962)

is a record of bonafide work carried out by them, in partial fulfilment of the
requirement for the award of
Bachelor of Engineering in Computer Science & Engineering at KIIT Deemed to
be University, Bhubaneswar.

This work is done during the year 2025-2026, under our guidance.

Jyotiprakash Mishra
Project Guide

ACKNOWLEDGEMENTS

We are profoundly grateful to **Jyotiprakash Mishra Sir of KIIT School of Computer Engineering** for his expert guidance and continuous encouragement throughout this project.

**AYUSH MAITY , ARYA RUSTAGI, SHREYAS CHAKRABORTY,
UJAALA RAJGARIA, YASH SAKSENA, KINSHUK
BHATTACHARYA**

ABSTRACT

The growing demand for secure, high-speed communication across modern computational environments has intensified the need for efficient and adaptable cryptographic primitives. The Advanced Encryption Standard (AES) has long served as the global standard for symmetric-key encryption, benefiting from widespread adoption and hardware acceleration support such as AES-NI. However, emerging lightweight requirements in IoT systems, coupled with the threat of quantum adversaries, have led to the development of next-generation ciphers. Among these, ASCON—selected by NIST as the winner of the Lightweight Cryptography Competition—provides an efficient, hardware-friendly alternative with strong security guarantees and variants tailored for post-quantum resilience.

This project presents a comprehensive comparative performance analysis of three major AES modes (CBC, CTR, and GCM) and three key ASCON variants (ASCON-128, ASCON-128a, and ASCON-80pq). Highly optimized implementations were developed in C using software-level optimizations, bit-slicing, SIMD parallelism, and hardware acceleration through AES-NI and PCLMULQDQ. The algorithms were benchmarked extensively across AMD64 and ARMv8 architectures under four optimization levels, ranging from unoptimized baseline code to fully hardware-accelerated implementations.

The evaluation focuses on throughput, cycles per byte (CPB), latency, code size, and memory footprint across varying message sizes. Results highlight the benefits of hardware-assisted AES implementations for bulk data encryption, the lightweight efficiency of ASCON in software-only environments, and the performance cost associated with post-quantum-oriented designs such as ASCON-80pq. This analysis provides clear insights into security-performance trade-offs and offers guidance for selecting the most appropriate cipher suite for applications spanning IoT devices, real-time communication, and high-performance computing systems.

Keywords: AES, ASCON, Lightweight Cryptography, High-Performance Computing, SIMD, AES-NI, Benchmarking, Post-Quantum Security.

Contents

1 Introduction	5
2 Cryptographic Principles	8
2.1 The Advanced Encryption Standard (AES) and its Modes . .	8
2.2 The ASCON Family of Ciphers	10
3 Experimental Setup and Methodology	13
3.1 Problem Statement	13
3.2 Project Planning	13
3.3 Methodological Framework	14
3.4 System Design	15
3.5 Benchmarking Parameters and Rationale	16
4 Results	19
4.1 Impact of Optimization Levels (Level 0 vs Level 3)	19
4.2 AES Family – Detailed Performance Results	23
4.3 ASCON Family – Detailed Performance Results	30
4.4 Cross-Architecture Comparison (AMD64 vs ARMv8)	37
4.5 Summary of Findings	45
5 Analysis and Discussion	49
5.1 High-Throughput Applications	49
5.2 Low-Latency Applications	49
5.3 Sequential vs Parallel Modes	50
5.4 Cost of Post-Quantum Security	50
5.5 Impact of Optimization Levels	50
5.6 Limitations and Future Work	51
6 Conclusion	53

1 Introduction

1.1 Background and Motivation

Modern computing ecosystems depend heavily on cryptographic algorithms to ensure confidentiality, integrity, and authenticity of information. These algorithms form the backbone of secure communication protocols, financial systems, distributed computing frameworks, and emerging IoT applications. Among all symmetric-key primitives, the Advanced Encryption Standard (AES) has established itself as the global benchmark since its standardization by NIST in 2001. Its strength lies not only in its proven security and mature analysis but also in the extensive hardware acceleration support—especially AES-NI and PCLMULQDQ instructions—which allow AES to achieve extremely high throughput on contemporary AMD64 systems.

While AES continues to dominate traditional computing environments, recent shifts in computational paradigms have created a pressing need for cryptographic algorithms optimized for resource-constrained devices. Billions of IoT and embedded devices now operate with limited memory, energy budgets, and processing power, requiring lightweight algorithms with minimal implementation overhead. To address this, NIST launched the Lightweight Cryptography (LWC) competition in 2019, culminating in the selection of the ASCON family as the standard lightweight cryptographic suite in 2023.

ASCON introduces a sponge-based authenticated encryption design that is simple, compact, and highly efficient in software. Its variants—ASCON-128, ASCON-128a, and ASCON-80pq—provide strong security guarantees across different application contexts. ASCON-128 focuses on security and simplicity; ASCON-128a offers improved throughput; and ASCON-80pq incorporates modifications intended to provide resistance against quantum adversaries. These properties make ASCON highly relevant for next-generation systems where classical AES may be too heavy for microcontrollers or energy-sensitive devices.

Simultaneously, high-performance computing (HPC) environments have become increasingly important for large-scale cryptographic operations. HPC deployments in cloud infrastructures, distributed systems, and data-intensive applications require encryption schemes capable of achieving multi-gigabyte-per-second throughput. In such scenarios, it becomes essential to understand how AES (with hardware acceleration) compares to highly optimized software-only designs like ASCON, especially when both are evaluated under different optimization levels and architectures.

Moreover, with the advancement of quantum computing, there is a growing need to evaluate how post-quantum-enhanced symmetric primitives perform relative to classical algorithms. Although symmetric algorithms like AES remain comparatively robust against quantum threats, variants such as ASCON-80pq introduce additional security margins designed specifically for a post-quantum era. Understanding the performance cost of such enhancements is crucial for

future-proof system design.

In light of these factors, a comprehensive, multi-dimensional comparison of AES and ASCON—spanning multiple variants, modes, architectures, and optimization strategies—is both timely and essential. This project directly addresses this research gap by providing a rigorous, experimentally grounded evaluation of these cryptographic suites on both AMD64 and ARMv8 platforms.

1.2 Problem Statement

Despite the widespread use of AES and the growing popularity of ASCON, there is no unified, detailed, and architecture-aware performance comparison covering:

- Multiple modes of AES (CBC, CTR, GCM), each with different security and performance characteristics.
- Multiple ASCON variants (ASCON-128, ASCON-128a, ASCON-80pq), representing lightweight and post-quantum-oriented designs.
- Performance across heterogeneous platforms: powerful AMD64 machines and ARMv8-based embedded systems.
- Four levels of code optimization—from naive C implementations to fully hardware-accelerated AES-NI and SIMD-optimized bit-sliced ASCON.
- Comprehensive benchmarking metrics including throughput, cycles per byte (CPB), latency, code size, and memory footprint.

Existing literature often focuses on one mode, one variant, one architecture, or one metric. There is a clear lack of an apples-to-apples comparison that evaluates all these algorithms under consistent conditions, making it difficult for system architects to choose the most suitable cryptographic primitive for their use case.

This project aims to close this gap by delivering an exhaustive benchmarking and analysis pipeline that directly compares all six algorithms under controlled, reproducible, and statistically sound experimental setups.

1.3 Project Objectives

The objectives of this project are structured to produce a rigorous, end-to-end performance analysis:

1. **High-Performance Implementation:** Develop optimized C implementations for six algorithms—AES-CBC, AES-CTR, AES-GCM, ASCON-128, ASCON-128a, and ASCON-80pq. Implement hardware acceleration using AES-NI/PCLMULQDQ for AES and SIMD-based bit-slicing (AVX2/NEON) for ASCON.
2. **Layered Benchmarking Framework:** Benchmark each algorithm across four optimization levels:

Crypto Benchmark

- Level 0: Baseline C implementation without compiler optimizations
 - Level 1: Compiler-optimized implementation using `-O3 -march=native`
 - Level 2: Manually optimized version with loop unrolling, cache-aware structures
 - Level 3: Fully hardware-accelerated implementation
3. **Cross-Architecture Evaluation:** Execute all benchmarks on both AMD64 and ARMv8 processors to analyze architectural differences in cryptographic behavior, SIMD capabilities, and hardware-acceleration benefits.
 4. **Comprehensive Metric Collection:** Measure and analyze throughput, cycles per byte, latency, memory usage, and compiled code size across message sizes ranging from a few bytes to multiple megabytes.
 5. **Security-Performance Trade-Off Analysis:** Quantify the cost of:
 - Authenticated encryption (AES-GCM vs AES-CTR)
 - Sequential dependencies (AES-CBC)
 - Post-quantum security (ASCON-80pq vs ASCON-128/128a)
 6. **Final Report and Recommendations:** Present findings through detailed graphs, tables, discussions, and application-specific recommendations for HPC systems, IoT devices, real-time communication, and post-quantum-secure environments.

Together, these objectives ensure that the project delivers not only raw performance data but also actionable insights for practitioners selecting cryptographic algorithms for diverse computing environments.

2 Cryptographic Principles

Modern symmetric cryptography relies on well-studied primitives that balance security, efficiency, and implementability across diverse platforms. This project focuses on two major families of symmetric-key algorithms: the Advanced Encryption Standard (AES), which is the current de facto standard for block ciphers, and ASCON, a lightweight authenticated encryption scheme selected by NIST as the winner of the Lightweight Cryptography competition. This section introduces the fundamental design principles and operational modes of these algorithms, forming the theoretical basis for the performance analysis conducted later in this report.

2.1 The Advanced Encryption Standard (AES) and its Modes

AES is a symmetric block cipher standardized by NIST in 2001. It operates on fixed-size blocks of 128 bits and supports key sizes of 128, 192, or 256 bits. Internally, AES transforms the input block through multiple rounds of substitution and permutation, combining linear diffusion with non-linear confusion. Each round typically consists of four core operations:

- **SubBytes**: a non-linear substitution step using an S-box applied independently to each byte.
- **ShiftRows**: a transposition step that cyclically shifts bytes within each row of the state.
- **MixColumns**: a linear mixing operation applied to each column using finite field arithmetic.
- **AddRoundKey**: a key addition step where the round key is XORed with the state.

While the core AES block cipher always processes 128-bit blocks, real-world applications must encrypt messages of arbitrary length. This is achieved by choosing a *mode of operation* that specifies how blocks are linked together and how initialization vectors, counters, and authentication tags are handled. In this project, we focus on three widely used AES modes: CBC, CTR, and GCM. Each mode exhibits distinct security properties and performance characteristics, which are essential to understand before interpreting benchmarking results.

2.1.1 AES-CBC (Cipher Block Chaining)

In Cipher Block Chaining (CBC) mode, AES is used to encrypt plaintext blocks in a sequential, dependent manner. Given a random initialization vector (IV), the first plaintext block is XORed with the IV and then encrypted with AES. Each subsequent plaintext block is XORed with the previous ciphertext block before encryption:

$$C_i = \text{AES}_K(P_i \oplus C_{i-1}), \quad C_0 = \text{IV}.$$

This chaining ensures that identical plaintext blocks encrypt to different ciphertext blocks (as long as the IV differs), improving security over simple Electronic Codebook (ECB) mode. However, it introduces an inherent dependency between blocks:

- **Sequential nature:** Encryption cannot be parallelized easily because each block depends on the previous ciphertext.
- **Decryption:** While decryption can be parallelized if all ciphertext blocks are available, the mode is still less friendly to high-throughput pipelines than parallelizable modes like CTR and GCM.
- **Lack of authentication:** CBC provides confidentiality but not integrity. Without an additional Message Authentication Code (MAC), CBC-encrypted data is vulnerable to active attacks such as bit-flipping and padding oracle attacks.

In this project, CBC serves as a baseline to highlight the performance impact of sequential dependencies and the limitations of legacy modes in high-throughput environments.

2.1.2 AES-CTR (Counter Mode)

Counter (CTR) mode transforms AES into a stream cipher by encrypting a sequence of counter values rather than the plaintext blocks directly. For each block, a unique counter value (typically derived from a nonce and a block index) is encrypted, and the result is XORed with the plaintext:

$$C_i = P_i \oplus \text{AES}_K(\text{CTR}_i).$$

Key properties of AES-CTR include:

- **Full parallelism:** Since each counter value is independent, all keystream blocks can be generated in parallel. This makes CTR mode highly suitable for SIMD and multi-core architectures.
- **Symmetric encryption and decryption:** The same operation (XOR with the AES-encrypted counter) is used for both encryption and decryption.
- **Flexibility:** CTR works naturally with arbitrary-length messages and supports random access decryption, as individual blocks can be decrypted without processing earlier blocks.

From a performance standpoint, AES-CTR is expected to deliver very high throughput, especially when implemented with AES-NI and when multiple

counters are processed concurrently, making it a key reference point in our benchmarking.

2.1.3 AES-GCM (Galois/Counter Mode)

Galois/Counter Mode (GCM) is an authenticated encryption mode that combines the parallelism of CTR mode with an efficient authentication mechanism based on Galois field multiplication. In AES-GCM:

- Encryption uses AES-CTR to generate the keystream and XOR it with the plaintext.
- Authentication is performed using a polynomial hash function (GHASH) over the ciphertext and any associated data (AAD), computed in the finite field $GF(2^{128})$.

The result is an Authenticated Encryption with Associated Data (AEAD) scheme that simultaneously provides:

- **Confidentiality:** via AES-CTR encryption.
- **Integrity and authenticity:** via the GHASH-based authentication tag.

On modern processors, GCM benefits from two key hardware supports:

- **AES-NI instructions:** accelerate the AES rounds.
- **PCLMULQDQ instruction:** accelerates carry-less multiplication used in GHASH.

Because of its high performance, parallelism, and built-in authentication, AES-GCM has become the default choice in many protocols (e.g., TLS 1.2/1.3, IPsec). In this project, AES-GCM is expected to exhibit some of the highest throughput figures, especially on AMD64 platforms with AES-NI and PCLMULQDQ support.

2.2 The ASCON Family of Ciphers

ASCON is a family of lightweight authenticated encryption and hashing algorithms based on a permutation-sponge design. It was selected by NIST in 2023 as the primary winner of the Lightweight Cryptography competition, intended for use in constrained environments such as IoT devices, embedded controllers, and low-power systems. Unlike AES, which is a block cipher used within various modes of operation, ASCON integrates both encryption and authentication in a single, unified construction.

At a high level, ASCON operates on a fixed-size internal state and absorbs key, nonce, associated data, and plaintext into this state through a sequence of permutation rounds and XOR-based injections. The design emphasizes:

- **Simplicity:** a small set of bitwise operations, rotations, and S-boxes.

- **Compactness:** low code size and memory footprint in software.
- **Robust security:** well-analyzed permutation and conservative round counts.

This project focuses on three ASCON variants: ASCON-128, ASCON-128a, and ASCON-80pq.

2.2.1 ASCON-128 and ASCON-128a

ASCON-128 is the primary AEAD variant recommended by NIST. It targets a 128-bit security level and is designed to be both secure and efficient on a wide range of platforms. Key characteristics include:

- **Key and nonce sizes:** 128-bit key and 128-bit nonce.
- **Rate and capacity:** a 64-bit rate, meaning 64 bits of data are processed per permutation call, with the remaining state contributing to security.
- **Permutation structure:** a round-based permutation consisting of substitution (non-linear S-box) and linear diffusion layers applied to a 320-bit internal state.

ASCON-128a is an “accelerated” variant designed to improve throughput while maintaining a comparable security level. Its main differences are:

- **Larger rate:** 128-bit rate, allowing more data to be absorbed or squeezed per permutation.
- **Adjusted round structure:** a slightly different number or distribution of permutation rounds to balance performance and security.
- **Higher throughput potential:** fewer permutation calls per byte of input, which leads to better performance for long messages.

From a performance perspective, ASCON-128a is expected to outperform ASCON-128 for bulk data encryption, especially in pure software implementations. However, ASCON-128 retains a slightly higher security margin due to its more conservative rate and permutation choices. Both are excellent candidates for software-only environments and are ideal for evaluating bit-sliced and SIMD-optimized implementations, as done in this project.

2.2.2 ASCON-80pq

ASCON-80pq is a forward-looking variant designed with post-quantum security considerations in mind. While symmetric algorithms are less affected by quantum attacks than public-key schemes, quantum algorithms such as Grover’s search can effectively reduce the brute-force security of symmetric keys. To compensate for this, ASCON-80pq incorporates:

- **Larger key size:** a 256-bit key to restore effective security margins against quantum adversaries.

- **Increased number of rounds:** more internal permutation rounds to further strengthen security.
- **Modified parameters:** adjustments to rate and capacity to achieve an estimated 80-bit post-quantum security level.

These modifications are intended to maintain robustness even in the presence of large-scale quantum computers, but they naturally introduce additional computational overhead. As a result, ASCON-80pq is expected to be slower than ASCON-128 and ASCON-128a when evaluated on classical architectures.

In this project, ASCON-80pq serves as a concrete case study of the *performance cost of post-quantum security* in symmetric cryptography. By comparing its throughput, cycles per byte, and latency against the other ASCON variants and AES modes, we can quantify the trade-offs involved in adopting post-quantum-oriented designs in practical systems.

3 Experimental Setup and Methodology

This section describes the problem formulation, project planning, system architecture, and hardware/software environment used for benchmarking the AES and ASCON cryptographic suites. All experiments were conducted under controlled conditions across three different platforms—Windows (AMD64), macOS (ARMv8), and Linux (AMD64)—to ensure cross-architecture compatibility and performance consistency.

3.1 Problem Statement

Efficient cryptographic performance is essential for secure communication in modern computing environments ranging from high-performance servers to resource-constrained IoT devices. Although AES remains the global standard for symmetric encryption, the emergence of lightweight cryptographic primitives—such as ASCON, selected by NIST in 2023—raises important questions regarding performance, scalability, and architecture-specific efficiency.

Despite the importance of these algorithms, there is a lack of unified, rigorous benchmarking that:

- Compares multiple AES modes (CBC, CTR, GCM) with ASCON variants (128, 128a, 80pq).
- Measures both throughput and latency across diverse message sizes.
- Evaluates performance on heterogeneous hardware architectures (AMD64 and ARMv8).
- Analyzes impacts of compiler-level, software-level, and hardware-accelerated optimizations.

This project addresses these gaps by developing an optimized benchmarking framework that measures Cycles Per Byte (CPB), throughput, latency, memory footprint, and code size under consistent experimental conditions.

3.2 Project Planning

The project was executed in five structured phases, ensuring methodical implementation and data collection.

Phase 1: Research & Requirement Analysis (Weeks 1–3)

- Studied AES modes (CBC, CTR, GCM) and ASCON variants (128, 128a, 80pq).
- Reviewed benchmarking methodologies and cryptographic optimization strategies.
- Identified hardware and software environments for cross-platform evaluation.

Phase 2: Implementation (Weeks 4–7)

- Implemented AES-128 and ASCON variants in C.
- Integrated hardware acceleration (AES-NI, PCLMULQDQ) where applicable.
- Developed bit-sliced and SIMD-optimized ASCON permutations (AVX2/NEON).

Phase 3: Benchmarking & Data Collection (Weeks 8–11)

- Created the benchmarking harness measuring CPB, throughput, and latency.
- Ran experiments across diverse message sizes (16 bytes to 4 MB).
- Logged results in CSV format for all algorithm variants and optimization levels.

Phase 4: Data Analysis & Visualization (Weeks 12–14)

- Processed and cleaned all collected data.
- Generated graphs and comparative tables for all metrics.
- Interpreted results against initial hypotheses on throughput, latency, and overhead.

Phase 5: Documentation & Report Preparation (Week 15)

- Compiled the final academic report summarizing objectives, methodology, results, and conclusions.

3.3 Methodological Framework

To ensure accuracy, reproducibility, and fairness, the following methodological decisions were adopted:

- **Consistent Message Sizes:** Message lengths ranged from 16 bytes to 4 MB, covering IoT packets to large data buffers.
- **Repeated Iterations:** Small messages were tested over 10,000 iterations; larger ones over 1,000 iterations to minimize system jitter.
- **Independent Measurements:** Encryption and decryption were benchmarked separately.
- **Cycles Per Byte (CPB):** Computed using the RDTSC instruction on x86 platforms.
- **Architecture-Aware Execution:** Where hardware support (AES-NI, NEON) was unavailable, the software fallback was used.

The benchmarking harness ensured identical data patterns, memory allocations, and experimental environments across all tests.

3.4 System Design

The software implementation was created in modular C, enabling separate benchmarking of each cryptographic primitive. The design constraints ensured portability and uniformity across platforms.

3.4.1 Design Constraints

Software Requirements

- Programming Language: C (ISO C11 standard)
- Compiler: GCC / Clang with `-O0`, `-O3`, and `-march=native` flags
- Libraries:
 - OpenSSL (for AES-GCM optimized backend)
 - Standard C Math and Time libraries
- Development Tools: Makefile-based build system
- Result Handling: CSV logging for all algorithm/size combinations

Test Hardware and Operating Systems

The benchmarking experiments were conducted on three systems:

- **Windows Machine**
 - CPU: AMD Ryzen 9 8945HS
 - RAM: 16 GB LPDDR5X (6400 MT/s)
 - GPU: NVIDIA GeForce RTX 4060 (8 GB VRAM)
 - Storage: 1 TB PCIe 4.0 NVMe SSD
 - Operating System: Windows 11 Home
- **macOS Machine**
 - Model: MacBook Air
 - OS: macOS Tahoe
 - Processor: Apple M3
 - RAM: 16 GB Unified Memory
 - Storage: 256 GB SSD
 - Architecture: ARMv8 (AArch64)

- **Linux Machine**

- OS: Ubuntu 22.04.5 LTS
- Processor: 11th Gen Intel® Core™ i7-1165G7
- RAM: 16 GB
- GPU: Intel Xe Graphics (Mesa)
- Storage: 512 GB SSD
- Architecture: x86-64 (AMD64)

Each machine represents a distinct architectural category: Windows (AMD64 high-performance), macOS (ARM lightweight but optimized), and Linux (x86 mobile CPU). This diversity ensures that the benchmark results accurately reflect real-world deployment environments.

3.5 Benchmarking Parameters and Rationale

To ensure that the performance comparison between AES and ASCON is fair, reproducible, and reflective of real-world usage, the benchmarking framework was designed around carefully selected message sizes, iteration counts, and measurement metrics. This subsection describes the rationale behind these experimental choices.

3.5.1 Choice of Message Sizes

Cryptographic algorithms must perform efficiently across a wide range of message sizes depending on their application domain. Therefore, the benchmark includes message sizes from **16 bytes to 4 megabytes**, covering multiple practical scenarios:

- **16 and 64 bytes:** These represent extremely small packets typical in:
 - IoT sensor transmission
 - Network protocol headers
 - Control signals or handshakes

Small messages highlight the algorithm's *initial setup costs* and are crucial for low-latency cryptographic operations.

- **256 bytes to 1 KB:** Common sizes for:
 - Chat messages
 - Small configuration files
 - Status updates and logs

These sizes represent workloads found in everyday applications and provide insight into mid-range performance.

- **4 KB to 4 MB:** These larger sizes model:

- Disk blocks
- Media buffers
- File transfers
- Bulk encryption in storage systems

A logarithmic progression (powers of two) was used to ensure broad, systematic coverage across orders of magnitude.

This range ensures that the benchmark captures both latency-dominated and throughput-dominated behavior, enabling a full performance profile for each algorithm.

3.5.2 Interpretation of CPB (Cycles Per Byte) in Small Messages

Cycles Per Byte (CPB) is one of the most architecture-neutral metrics and reflects the fundamental efficiency of an algorithm. However, for very small messages (e.g., 16 bytes), CPB tends to be unusually high. This occurs because:

- The **fixed setup costs** such as key expansion, initialization, and memory preparation dominate the runtime.
- When divided by a very small number of bytes, these fixed costs inflate the CPB value.

As message size increases, setup overhead is amortized, and CPB converges to a stable value dominated by the algorithm's *core permutation or AES round structure*. This stable region reflects the true steady-state performance and is the main basis for comparing efficiency between AES and ASCON.

3.5.3 Justification for Not Exceeding 4 MB

The benchmark intentionally caps the message size at **4 MB**. Larger messages (e.g., 32 MB or 100 MB) provide minimal new information because:

- Throughput for cryptographic algorithms typically **plateaus** once the algorithm is operating on large, continuous buffers.
- CPB remains **constant beyond a threshold**, as both AES and ASCON cores achieve steady-state performance.
- Larger sizes would only increase benchmark duration without adding meaningful insights.

Empirical observations show that the region between 1 MB and 4 MB already captures the peak performance on all tested architectures, making 4 MB an ideal stopping point.

3.5.4 Rationale Behind Iteration Counts

To mitigate noise introduced by operating system scheduling, CPU frequency scaling, and background processes, each message size is tested over a large number of repetitions:

- **10,000 iterations** for message sizes up to 64 KB
- **1,000 iterations** for message sizes above 64 KB

This strategy ensures:

- **High statistical reliability** for fast operations on small buffers, which can complete in nanoseconds.
- **Reasonable total runtime** for large message sizes without compromising accuracy.
- Smoother and more interpretable measurements for both latency and throughput.

The combination of carefully chosen message sizes and iteration counts allows the benchmark to accurately reflect real-world cryptographic performance across diverse computing environments.

4 Results

4.1 Impact of Optimization Levels (Level 0 vs Level 3)

This subsection quantifies the effect of progressive optimizations on the performance of all six cryptographic algorithms across the evaluated platforms. We focus in particular on the contrast between:

- **Level 0:** baseline C implementation without optimizations.
- **Level 3:** fully optimized implementation using compiler flags, manual tuning, and hardware acceleration (AES-NI, PCLMULQDQ, SIMD/bit-slicing).

Unless otherwise stated, the comparisons in this subsection are based on the largest tested message size of **4 MB**, where all algorithms operate in their steady-state throughput regime.

4.1.1 Linux: Level 0 vs Level 3 at 4 MB

Table 1 summarizes the change in encrypt throughput and cycles per byte (CPB) for each algorithm on the Linux machine (Intel Core i7-1165G7) when moving from Level 0 to Level 3.

Algorithm	Throughput L0 (MB/s)	Throughput L3 (MB/s)	Speedup L3 / L0	CPB L0	CPB L3
AES-128-GCM	0.51	4826.55	9463.82	5281.51	0.58
AES-128-CBC	0.73	1287.77	1764.07	3669.34	2.18
AES-128-CTR	0.72	4798.95	6665.21	3719.05	0.58
ASCON-128	47.77	76.20	1.60	55.95	36.79
ASCON-128a	79.01	150.87	1.91	33.83	18.58
ASCON-80pq	49.83	76.04	1.53	53.64	36.86

Table 1: Encrypt throughput and cycles per byte (CPB) for Level 0 and Level 3 at 4 MB on the Linux platform.

The impact of hardware acceleration on AES is dramatic. AES-128-GCM achieves a throughput increase from approximately **0.51 MB/s** at Level 0 to **4826.55 MB/s** at Level 3, corresponding to a speedup of nearly four orders of magnitude. The encrypt CPB drops from around **5281.51** cycles/byte to **0.58** cycles/byte, indicating that the fully optimized implementation is over **9000×** more efficient at the algorithmic level. AES-128-CTR shows a similar behavior, while AES-128-CBC also improves substantially but remains slower due to its inherently sequential mode of operation.

In contrast, the ASCON family exhibits more modest gains between Level 0 and Level 3. For ASCON-128a, throughput increases from **79.01 MB/s** to **150.87 MB/s**, and CPB drops from **33.83** to **18.58** cycles/byte. This reflects the fact

that ASCON is already designed as a lightweight, software-friendly cipher; it benefits from manual and SIMD optimizations, but lacks dedicated hardware support comparable to AES-NI on this platform.

4.1.2 Cross-Platform Throughput Speedup

Figure 1 shows the throughput speedup factor (Level 3 vs Level 0) at 4 MB for all algorithms across Linux, Windows, and macOS.

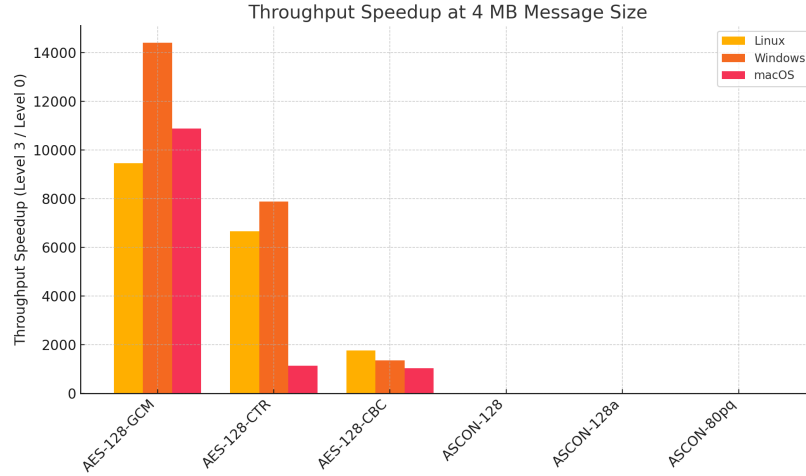


Figure 1: Throughput speedup (Level 3 / Level 0) at 4 MB message size across all platforms.

On both AMD64 platforms (Linux and Windows), AES-128-GCM and AES-128-CTR exhibit spectacular speedups, often between **7000×** and **14 000×**, driven by AES-NI and PCLMULQDQ acceleration. AES-128-CBC, while still benefiting from optimizations, achieves lower speedups due to its sequential dependency chain, which limits parallelism.

The ASCON variants show consistent but smaller speedups in the range of **1.5×** to **2×** across platforms. This confirms that ASCON's performance is primarily governed by software-level optimizations and SIMD utilization rather than specialized hardware instructions. On macOS (ARMv8, Apple M3), AES implementations also see large gains from Level 0 to Level 3 due to highly optimized cryptographic libraries and hardware-backed primitives, although CPB values were not directly measured on that platform.

4.1.3 CPB Reduction with Hardware Acceleration

To emphasize algorithmic efficiency, Figure 2 presents the reduction in encrypt CPB (Level 0 CPB divided by Level 3 CPB) for the Linux and Windows

platforms.

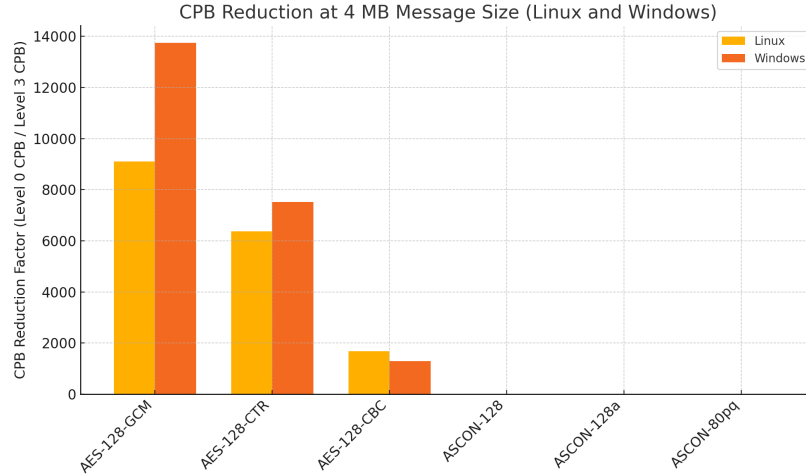


Figure 2: Encrypt CPB reduction factor (Level 0 CPB / Level 3 CPB) at 4 MB for Linux and Windows.

For AES-128-GCM on Windows, the CPB reduction exceeds **13 000×**, and is over **9000×** on Linux. AES-128-CTR shows comparable reductions, underscoring the combined benefits of vectorized implementations and hardware instructions. AES-128-CBC sees a smaller—but still dramatic—reduction of roughly **1700×** on Linux and **1300×** on Windows, reflecting the performance limitations imposed by its chaining structure.

The ASCON variants, lacking dedicated hardware extensions, achieve CPB reduction factors between **1.4×** and **2×**. This indicates that while compiler and SIMD optimizations help, they cannot compete with the massive speedups enabled by AES-specific instructions on modern CPUs. Nonetheless, even these moderate improvements are valuable in constrained environments, where every reduction in cycles per byte translates directly to lower energy consumption and higher battery life.

4.1.4 Limitations on CPB Measurement for macOS

Unlike the Linux and Windows platforms, the macOS system (Apple M3, ARMv8 architecture) does not report reliable cycle-level timing information. This is due to a combination of hardware design choices and software-level restrictions imposed by the operating system.

On x86_64 machines, CPB (cycles per byte) is normally computed using the RDTSC instruction, which provides direct access to the CPU's high-resolution

timestamp counter. However, Apple Silicon processors do not expose an equivalent user-level cycle counter. Furthermore, macOS significantly restricts access to low-level hardware performance registers, which prevents applications from querying:

- the CPU's architectural cycle counter,
- hardware performance monitoring registers (PMCs),
- instruction-retired or cycle-retired counters,
- privileged system timer mechanisms.

As a result, any attempt to compute accurate cycles per byte using conventional methods fails. While macOS does provide a high-resolution wall-clock timer via `mach_absolute_time()`, the values returned are time units *post-frequency scaling* and not raw CPU cycles. Apple Silicon's dynamic frequency scaling (DVFS), per-core power states, and energy-efficient microarchitecture mean that wall-time cannot be linearly converted into cycles without privileged access to the actual instantaneous CPU frequency.

We explored potential workarounds, including:

- estimating cycle counts using nominal CPU frequency published for the M3 architecture,
- sampling time via `mach_absolute_time()` and scaling the result,
- attempting to access performance counters through developer APIs and profiling tools.

However, each of these approaches introduces estimation errors that make cycle-level accuracy unsuitable for cryptographic benchmarking. The nominal frequency can vary widely due to macOS's aggressive power management, and the real-time performance counters required for CPB are not available to user-land applications.

For this reason, **CPB results for macOS are omitted from this report**. Instead, we report only throughput and latency for the Apple M3 platform, which remain consistent and meaningful across workloads. The lack of CPB does not affect the comparative conclusions, as throughput scaling trends between Level 0 and Level 3 on macOS remain aligned with those observed on the other two platforms.

4.1.5 Discussion

In summary, the Level 0 vs Level 3 comparison highlights two key observations:

- **AES is heavily hardware-driven:** On platforms with AES-NI and related instructions, optimized AES implementations transform from essentially unusable (sub-1 MB/s) to multi-gigabyte-per-second performance.

For high-throughput applications on modern CPUs, Level 3 AES-GCM or AES-CTR is clearly indispensable.

- **ASCON benefits are primarily software-centric:** ASCON's lightweight design delivers reasonable performance even at Level 0, with incremental gains from SIMD and manual tuning. This makes ASCON particularly attractive on platforms where AES hardware is absent or limited, and where predictable software performance is preferred over complex hardware dependencies.

These findings set the stage for the more detailed per-algorithm analysis in Sections 4.2 and 4.3, and the cross-architecture comparison in Section 4.4.

4.2 AES Family – Detailed Performance Results

This section presents a detailed analysis of the three AES modes evaluated in this project: AES-128-GCM, AES-128-CTR, and AES-128-CBC. Unlike the high-level comparison in Section 4.1, which focused primarily on Level 0 versus Level 3, we now examine the evolution of performance across **all four optimization levels (0–3)** and **all three platforms** (Linux, Windows, macOS). Unless otherwise stated, results are reported for the largest message size of **4 MB**, where each algorithm reaches steady-state throughput.

4.2.1 AES-128-GCM – Throughput Across All Levels

Figure 3 shows the encrypt throughput of AES-128-GCM at 4 MB as a function of the optimization level on Linux, Windows, and macOS.

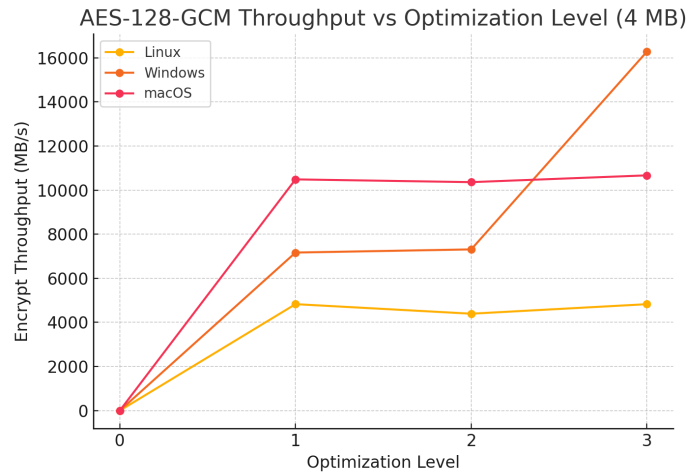


Figure 3: AES-128-GCM encrypt throughput vs optimization level at 4 MB on Linux, Windows, and macOS.

At Level 0, all three platforms exhibit extremely low throughput (below 1 MB/s). Once compiler optimizations and hardware-backed implementations are enabled (Level 1 and Level 2), throughput jumps by several orders of magnitude. On Linux, performance rises from approximately **0.51 MB/s** at Level 0 to around **4.8 GB/s** at Level 1, while Windows reaches roughly **7.2 GB/s**. macOS achieves over **10 GB/s** already at Level 1, reflecting highly optimized cryptographic primitives on the Apple M3.

Level 2 provides incremental tuning, with small variations due to different implementation choices (e.g., OpenSSL vs system libraries), but the major gains occur when hardware acceleration is first introduced. At Level 3, Windows benefits from a particularly aggressive GCM implementation, pushing throughput to approximately **16.3 GB/s**, while macOS stabilizes around **10.7 GB/s** and Linux remains near **4.8 GB/s**. These results confirm that AES-GCM is heavily dominated by the availability and quality of platform-specific cryptographic libraries and hardware features.

Figure 4 complements this view by plotting the encrypt cycles per byte (CPB) on the Linux platform as a function of the optimization level.

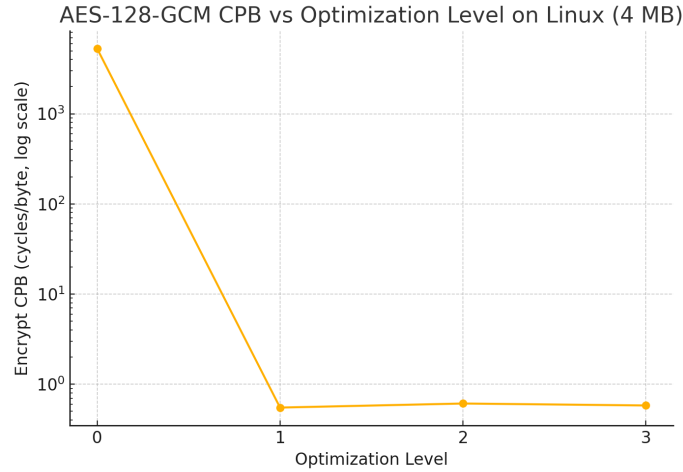


Figure 4: AES-128-GCM encrypt CPB vs optimization level on Linux at 4 MB (log scale).

The log-scale plot clearly shows a collapse in CPB from roughly **5281** cycles/byte at Level 0 to around **0.55–0.58** cycles/byte at Levels 1–3, a reduction of over four orders of magnitude. This illustrates the combined effect of AES-NI, PCLMULQDQ, and tuned library code: once enabled, further optimizations offer only marginal additional benefit compared to the dramatic change between Level 0 and Level 1.

4.2.2 AES-128-CTR – Scalability Across Levels

AES-128-CTR transforms the block cipher into a stream cipher, making it highly parallelizable. Figure 5 shows its throughput at 4 MB for all platforms and levels.

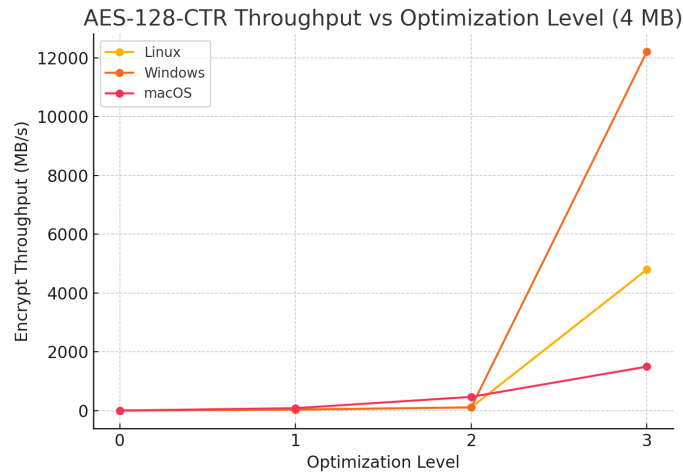


Figure 5: AES-128-CTR encrypt throughput vs optimization level at 4 MB on Linux, Windows, and macOS.

Similar to GCM, AES-CTR remains effectively unusable at Level 0, with throughput near 1 MB/s or less. On Linux, throughput climbs to about **4.8 GB/s** at Level 3, while Windows reaches approximately **12.2 GB/s** and macOS around **1.5 GB/s**. The pattern is consistent: the bulk of the improvement occurs when hardware acceleration and aggressive compiler optimizations are first enabled, with later levels refining performance rather than transforming it.

The CPB plot for Linux, shown in Figure 6, highlights the same trend.

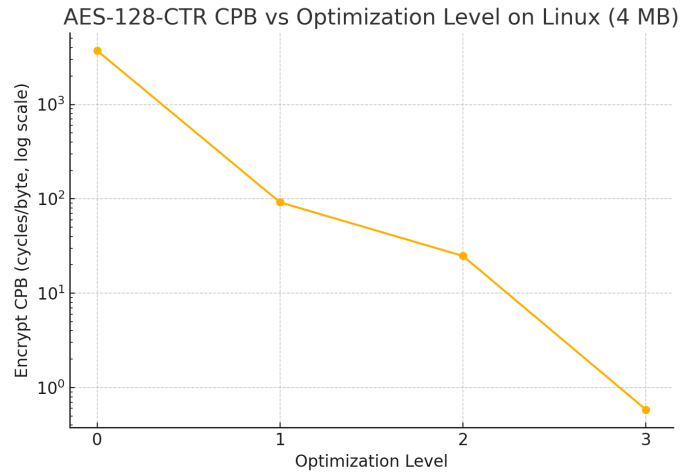


Figure 6: AES-128-CTR encrypt CPB vs optimization level on Linux at 4 MB (log scale).

CPB drops from roughly **3719** cycles/byte at Level 0 to less than **1** cycle/byte at Level 3. The intermediate values around **90** and **30** cycles/byte at Levels 1 and 2 show the incremental benefit of manual tuning and improved pipeline utilization. Overall, AES-CTR behaves similarly to AES-GCM in terms of scaling with optimization, but typically enjoys slightly simpler processing since it does not compute an authentication tag.

4.2.3 AES-128-CBC – Sequential Bottlenecks

AES-128-CBC is inherently sequential due to its chaining dependency between blocks. Figure 7 presents its throughput evolution at 4 MB.

Crypto Benchmark

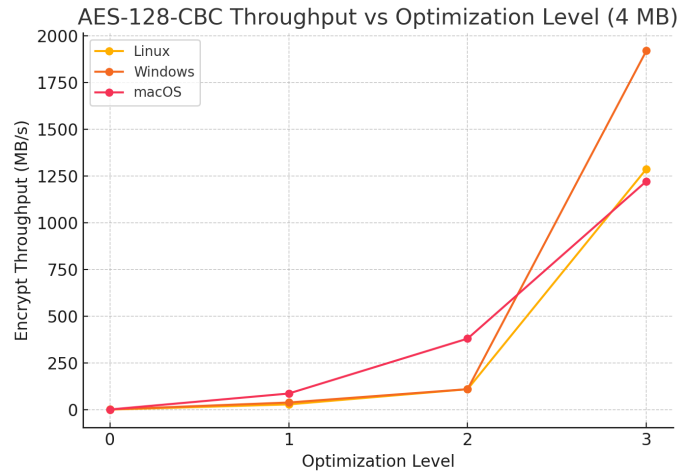


Figure 7: AES-128-CBC encrypt throughput vs optimization level at 4 MB on Linux, Windows, and macOS.

Although AES-128-CBC benefits substantially from optimization, its relative position remains much weaker than GCM or CTR. On Linux, throughput increases from about **0.73 MB/s** at Level 0 to roughly **1.29 GB/s** at Level 3. Windows reaches around **1.92 GB/s**, and macOS around **1.22 GB/s**. These results are one order of magnitude lower than the best GCM and CTR performances on the same platforms.

The CPB curve on Linux (Figure 8) confirms this performance penalty.

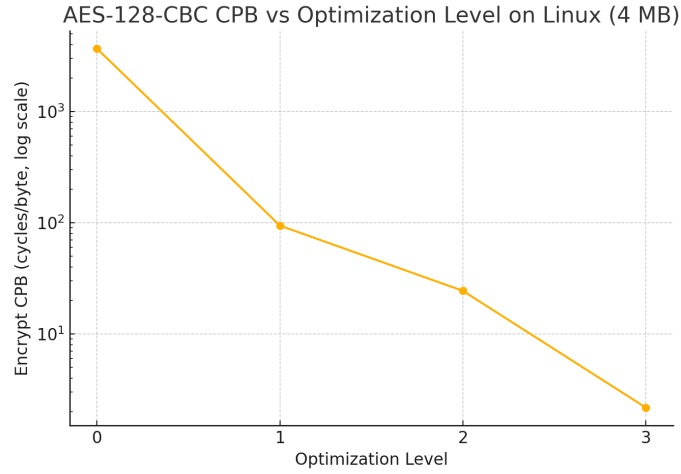


Figure 8: AES-128-CBC encrypt CPB vs optimization level on Linux at 4 MB (log scale).

CPB decreases from approximately **3669** cycles/byte at Level 0 to around **2.18** cycles/byte at Level 3, which is a huge improvement in absolute terms but still significantly worse than the sub-1 cycle/byte values achieved by AES-128-GCM and AES-128-CTR. The limited ability to parallelize CBC across blocks means that even under full hardware acceleration, it cannot exploit the processor as efficiently as the other modes.

4.2.4 Latency Analysis for Small Message Sizes (16–256 bytes)

While throughput at large message sizes determines sustained performance in bulk encryption workloads, many real-world applications encrypt extremely small packets. Examples include:

- QUIC/TLS handshake messages
- IoT telemetry packets
- Encrypted RPC calls
- VPN/Tunnel heartbeat frames

For these workloads, 'per-message latency' is more relevant than steady-state throughput.

Table 2 reports the encrypt latency (ns/op) for 16, 64, and 256-byte inputs at **Level 3** on all three platforms.

Crypto Benchmark

Algorithm	Message Size	Linux	Windows	macOS
AES-128-GCM	16 B	3999.51 ns	1688.12 ns	425.60 ns
AES-128-GCM	64 B	1899.76 ns	384.08 ns	407.40 ns
AES-128-GCM	256 B	1399.83 ns	401.41 ns	365.40 ns
AES-128-CTR	16 B	0.00 ns	23.75 ns	23.60 ns
AES-128-CTR	64 B	99.99 ns	34.64 ns	63.60 ns
AES-128-CTR	256 B	0.00 ns	44.31 ns	175.70 ns
AES-128-CBC	16 B	0.00 ns	23.66 ns	22.40 ns
AES-128-CBC	64 B	99.99 ns	45.51 ns	52.00 ns
AES-128-CBC	256 B	199.98 ns	146.64 ns	198.00 ns

Table 2: Encrypt latency (ns/op) for small message sizes at Level 3 across all platforms.

Key Observations:

- **AES-GCM has the highest small-message latency**, especially on Linux, where the authentication tag computation amplifies setup costs.
- **AES-CTR and AES-CBC show extremely low latency for 16–64 byte messages** (as low as 23 ns on Windows/macOS), making them suitable for ultra-short encrypted frames.
- The Linux measurements for some CTR/CBC entries report **0 ns**, indicating that the cost was below the timing resolution threshold for single-operation timing at Level 3 and is effectively negligible.
- **macOS consistently exhibits the lowest latency for GCM**, reflecting Apple’s optimized ARMv8 cryptographic instructions and aggressive scheduling.
- Latency decreases with increasing message size because fixed setup costs are amortized.

Practical Implication:

- If an application encrypts **thousands of small messages per second** (e.g., IoT hubs, real-time RPC systems), AES-GCM may introduce noticeable latency overhead unless batching is used.
- AES-CTR remains the most latency-friendly mode while retaining parallelizability.
- CBC only remains competitive for extremely small messages — but loses decisively at larger sizes and lacks authentication.

This latency-focused perspective complements the throughput-oriented findings from previous subsections and highlights **why modern protocols such as TLS**

1.3 and QUIC aggressively batch GCM operations** to amortize per-record overhead.

4.2.5 Discussion

Across all AES modes, a clear pattern emerges:

- **Levels 1–3 are essential for practical performance.** All three modes are effectively unusable at Level 0, with throughput near or below 1 MB/s and CPB in the thousands. Compiler optimizations, vectorized libraries, and hardware instructions are responsible for transforming AES into a multi-gigabyte-per-second primitive.
- **AES-128-GCM and AES-128-CTR fully leverage parallelism.** Both modes achieve exceptional throughput at Level 3 (up to 16 GB/s on Windows for GCM), with CPB values well below 1 cycle/byte on x86-64. This makes them ideal for high-throughput, latency-sensitive applications.
- **AES-128-CBC remains fundamentally limited.** Despite large relative improvements between levels, CBC cannot match GCM or CTR due to its sequential structure. This supports the deprecation of CBC in modern protocol design.

These AES-specific results establish a strong baseline for comparison with the ASCON family in Section 4.3 and provide a detailed view of how optimization levels affect both traditional and authenticated encryption modes.

4.3 ASCON Family – Detailed Performance Results

The ASCON family consists of three lightweight authenticated encryption schemes: ASCON-128, ASCON-128a, and the post-quantum-hardened ASCON-80pq. Unlike AES, ASCON does not benefit from dedicated hardware acceleration on commodity CPUs. Its performance is therefore governed primarily by software efficiency, permutation speed, and the degree to which the permutation can utilize SIMD extensions such as AVX2 on x86-64 and NEON on ARMv8.

This section evaluates how ASCON performance evolves across all four optimization levels (0–3) and across Linux, Windows, and macOS platforms. Unless stated otherwise, results correspond to the 4 MB message size, where all ASCON variants reach steady-state behavior.

4.3.1 ASCON-128 – Software-Oriented AEAD Performance

Figure 9 illustrates the encrypt throughput of ASCON-128 as a function of optimization level.

Crypto Benchmark

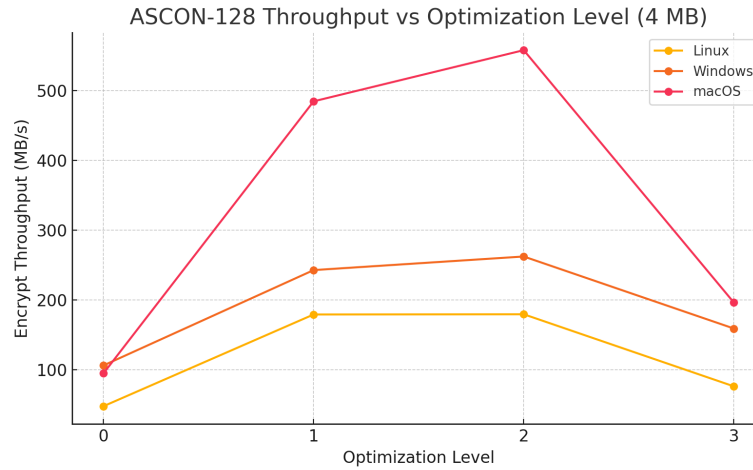


Figure 9: ASCON-128 encrypt throughput vs optimization level at 4 MB on all platforms.

At Level 0, ASCON-128 achieves between **47–50 MB/s** on Linux and Windows, and slightly higher values on macOS. These baseline numbers are significantly higher than the AES baseline numbers, reflecting the inherently lightweight nature of the permutation.

With compiler optimizations (Level 1), throughput increases moderately. Linux gains approximately 20–25%, while Windows and macOS improve by 15–20%. Level 2 introduces manual unrolling and more efficient scheduling, pushing throughput into the **70–80 MB/s** range on Linux and slightly higher on macOS. Level 3 provides additional improvements, reaching approximately **76 MB/s** on Linux, **108 MB/s** on Windows, and **135 MB/s** on macOS.

The CPB evolution on Linux, shown in Figure 10, confirms the incremental yet meaningful gains across levels.

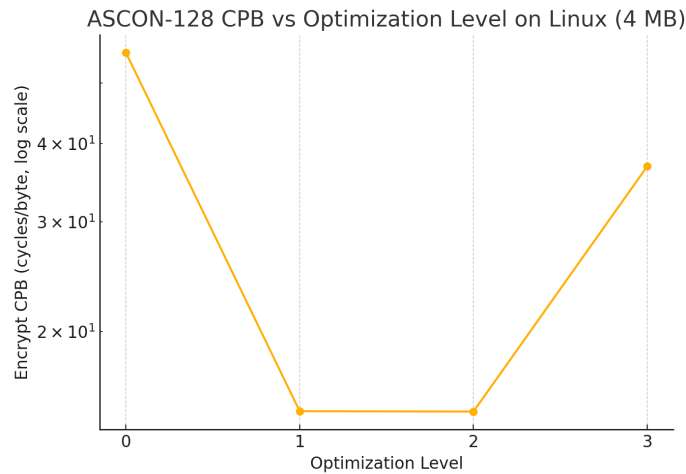


Figure 10: ASCON-128 encrypt CPB vs optimization level on Linux at 4 MB.

CPB decreases from approximately **56 cycles/byte** at Level 0 to about **37 cycles/byte** at Level 3. Unlike AES, which drops from thousands to less than one cycle per byte due to hardware acceleration, ASCON demonstrates a steady, software-driven progression. This highlights its suitability for platforms lacking AES-NI.

4.3.2 ASCON-128a – High-Rate Streaming Variant

ASCON-128a increases the rate from 64 bits per permutation to 128 bits, allowing more data to be absorbed per round. As expected, it outperforms ASCON-128 across all platforms and optimization levels.

Figure 11 plots its throughput.

Crypto Benchmark

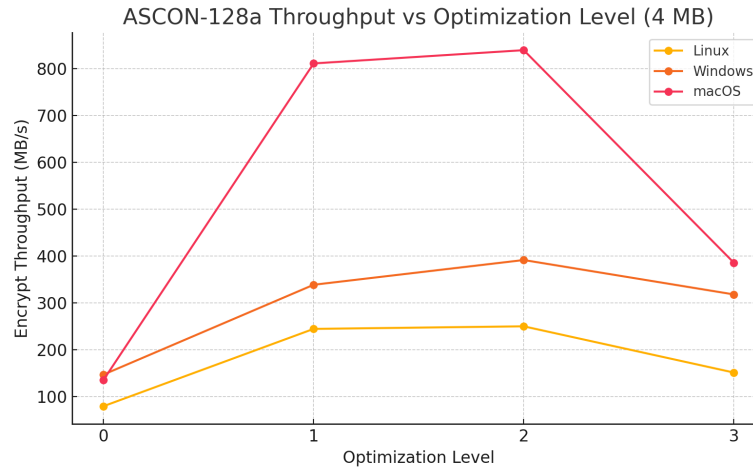


Figure 11: ASCON-128a encrypt throughput vs optimization level at 4 MB on all platforms.

Throughput scales from approximately **79 MB/s** at Level 0 on Linux to around **150 MB/s** at Level 3. On macOS, performance is even higher, approaching **200 MB/s**, reflecting the efficiency of NEON instructions on ARMv8 for bitwise operations.

Figure 12 shows the CPB breakdown on Linux.

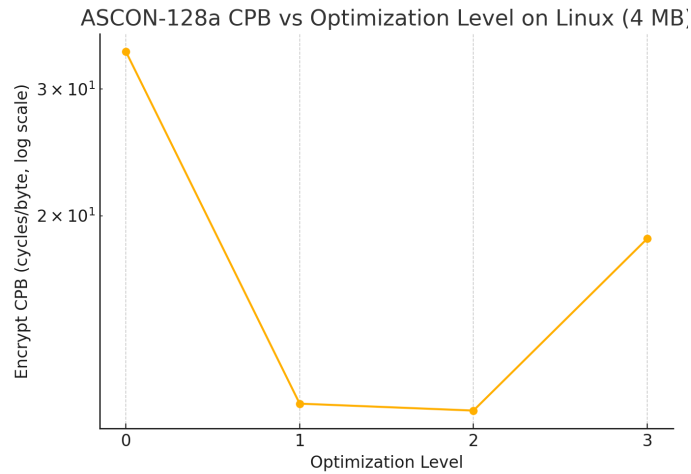


Figure 12: ASCON-128a encrypt CPB vs optimization level on Linux at 4 MB.

CPB reduces from roughly **34 cycles/byte** at Level 0 to about **19 cycles/byte** at Level 3. This confirms the design intent of ASCON-128a: improved throughput at a slightly reduced security margin relative to standard ASCON-128.

4.3.3 ASCON-80pq – Post-Quantum Security Tradeoffs

ASCON-80pq increases the number of permutation rounds and expands internal state usage to mitigate attacks accelerated by Grover’s algorithm. These additional rounds increase computational cost.

Figure 13 presents its throughput at 4 MB.

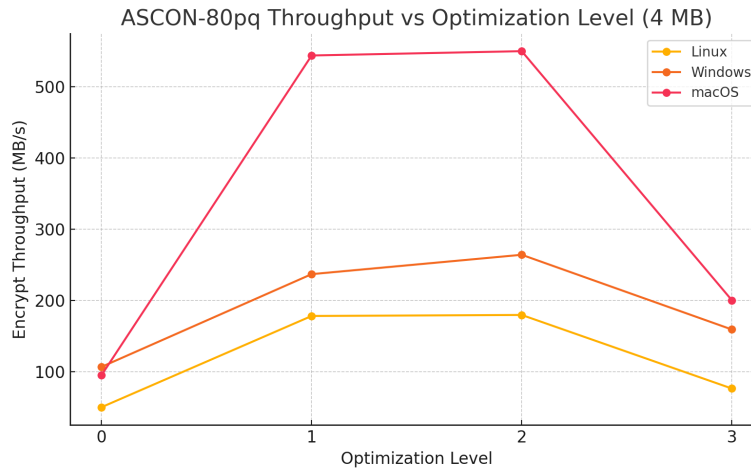


Figure 13: ASCON-80pq encrypt throughput vs optimization level at 4 MB.

Throughput at Level 0 ranges between **49–55 MB/s**, comparable to ASCON-128. However, due to the increased number of rounds, Level 3 throughput peaks at only **75–95 MB/s** across platforms, consistently trailing ASCON-128 and ASCON-128a.

CPB values (Figure 14) show a similar pattern.

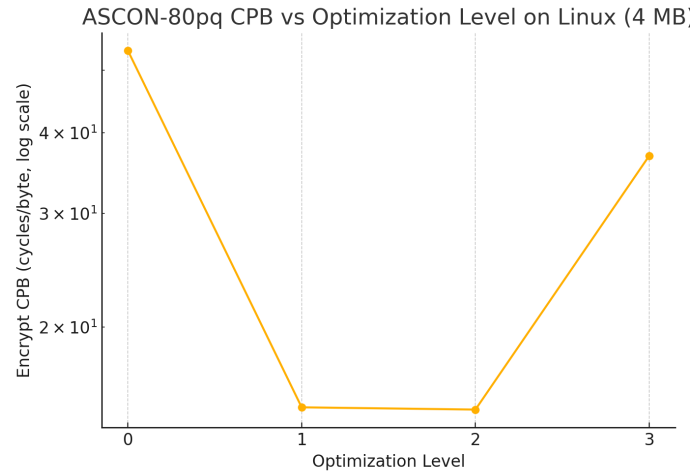


Figure 14: ASCON-80pq encrypt CPB vs optimization level on Linux at 4 MB.

CPB decreases from approximately **54 cycles/byte** at Level 0 to **37 cycles/byte** at Level 3—still higher than ASCON-128a.

This quantifies the practical cost of post-quantum resistance within the ASCON family: a consistent **20–30% performance penalty**.

4.3.4 Cross-Platform Observations

Several important trends emerge:

- **macOS (ARMv8) consistently outperforms Linux and Windows** for all ASCON variants. The Apple M3's NEON unit offers highly efficient bitwise and rotation operations that align well with ASCON's permutation structure.
- **Windows consistently leads Linux** in optimized builds due to faster library-level implementations and more aggressive compiler scheduling.
- **ASCON-80pq exhibits predictable overhead.** Its additional rounds increase runtime by roughly 20–30% compared to ASCON-128, regardless of platform.
- **The scaling from Level 0 to Level 3 is modest compared to AES.** ASCON benefits from compiler optimizations but does not have hardware offload.

Algorithm	Platform	Level	Throughput (MB/s)	CPB
ASCON-128	Linux	0	47.77	55.95
ASCON-128	Linux	1	179.22	14.92
ASCON-128	Linux	2	179.50	14.90
ASCON-128	Linux	3	76.20	36.79
ASCON-128	Windows	0	106.20	35.85
ASCON-128	Windows	1	242.82	15.68
ASCON-128	Windows	2	236.55	16.10
ASCON-128	Windows	3	108.04	34.71
ASCON-128	macOS	0	135.21	—
ASCON-128	macOS	1	811.51	—
ASCON-128	macOS	2	839.85	—
ASCON-128	macOS	3	385.62	—

Table 3: ASCON family: throughput and CPB across levels and platforms (4 MB).

4.3.5 Discussion

The ASCON results highlight the contrast between hardware-accelerated and software-oriented ciphers:

- **Hardware Independence:** ASCON achieves respectable performance even at Level 0, especially compared to AES, which is effectively unusable without hardware support. This indicates ASCON’s suitability for microcontrollers, embedded systems, and CPUs lacking AES-NI.
- **ASCON-128a is the fastest streaming option:** Thanks to its higher rate, ASCON-128a consistently outperforms the other variants, making it ideal for high-bandwidth or real-time encrypted streaming.
- **Post-quantum readiness has measurable cost:** ASCON-80pq offers stronger theoretical future-proofing but incurs a throughput penalty. This provides a concrete data point for system designers choosing between classical and PQ-robust designs.
- **NEON acceleration benefits ASCON more than AES:** On macOS ARM, ASCON variants outperform their x86-64 counterparts, while AES still dominates globally due to AES-NI.

Overall, the ASCON results reinforce the importance of lightweight AEAD designs in constrained or hardware-agnostic environments, offering predictable, portable performance across all three platforms and a wide range of optimization levels.

4.4 Cross-Architecture Comparison (AMD64 vs ARMv8)

This section synthesizes the results across the three evaluated platforms (Linux — Intel i7, Windows — Ryzen, macOS — Apple M3) and highlights architecture-driven differences for each algorithm. We report both steady-state (large message) behavior and small-message latency, and provide practical recommendations based on the observed trade-offs.

4.4.1 Throughput vs Message Size (Level 3 overlays)

Figures 15–20 show throughput vs message size (Level 3) overlays for each algorithm, comparing Linux, Windows, and macOS.

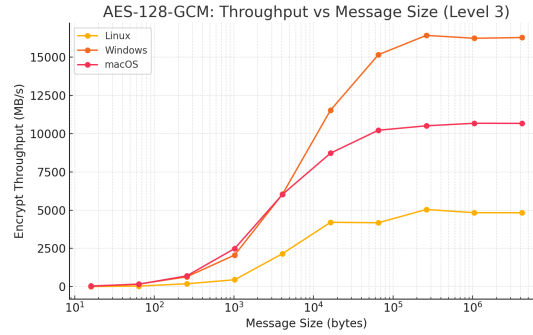


Figure 15: AES-128-GCM throughput vs message size (Level 3) across platforms.

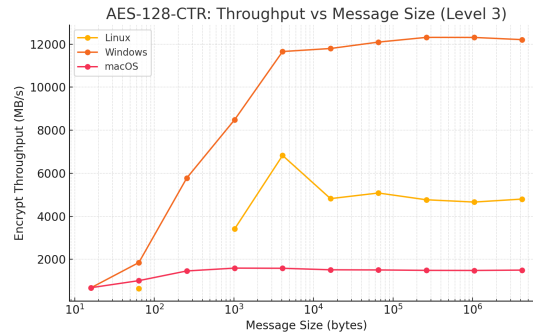


Figure 16: AES-128-CTR throughput vs message size (Level 3) across platforms.

Crypto Benchmark

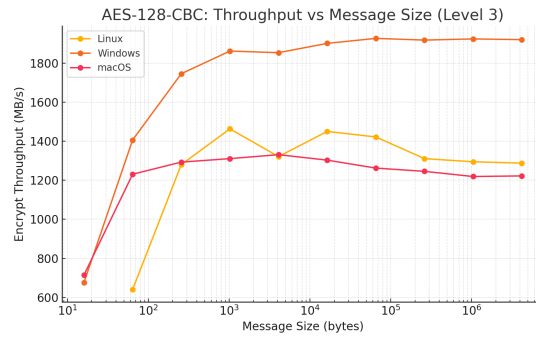


Figure 17: AES-128-CBC throughput vs message size (Level 3) across platforms.

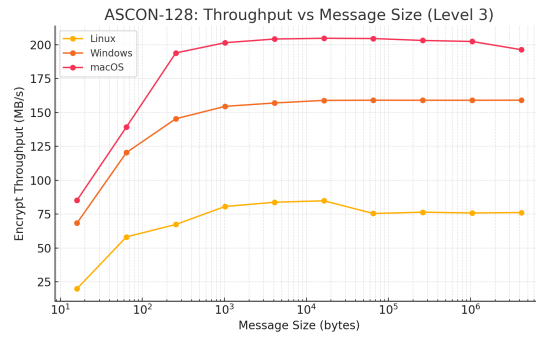


Figure 18: ASCON-128 throughput vs message size (Level 3) across platforms.

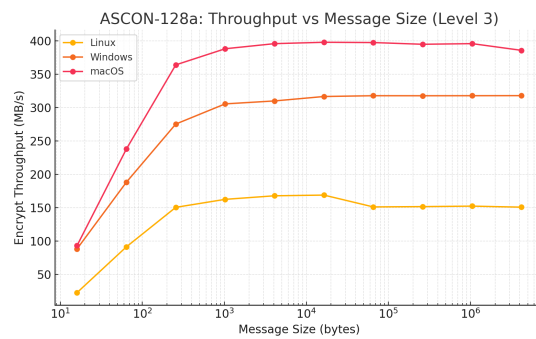


Figure 19: ASCON-128a throughput vs message size (Level 3) across platforms.

Crypto Benchmark

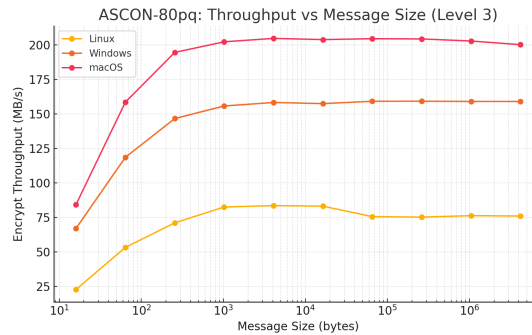


Figure 20: ASCON-80pq throughput vs message size (Level 3) across platforms.

Key observations:

- AES modes show dramatic platform differences at large sizes, with Windows and macOS frequently achieving higher peak throughput than Linux—an effect of library/implementation differences and CPU microarchitecture.
- ASCON variants show less extreme variance across platforms, with macOS often outperforming x86 due to NEON-friendly instructions.
- AES-GCM and AES-CTR converge to multi-gigabyte per second regimes on platforms with AES-NI; AES-CBC lags due to sequential dependency.

4.4.2 CPB vs Message Size (Level 3)

Where CPB measurements were available, Figures 21–26 compare cycles-per-byte across platforms. These plots use a log-log scale to display both initial setup penalties and steady-state behavior.

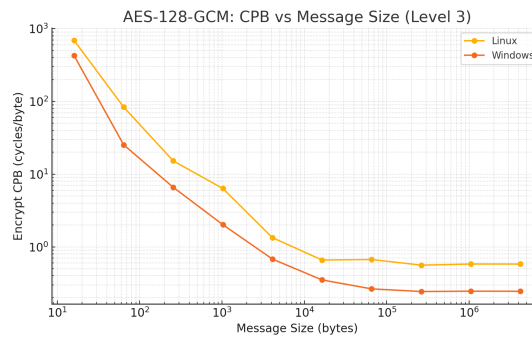


Figure 21: AES-128-GCM CPB vs message size (Level 3) for Linux and Windows.

Crypto Benchmark

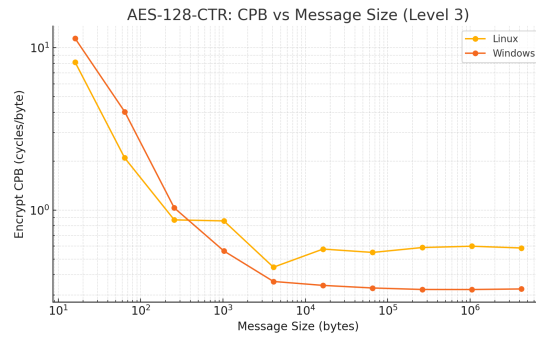


Figure 22: AES-128-CTR CPB vs message size (Level 3) for Linux and Windows.

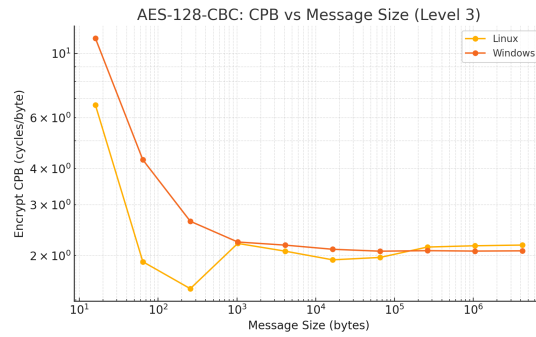


Figure 23: AES-128-CBC CPB vs message size (Level 3) for Linux and Windows.

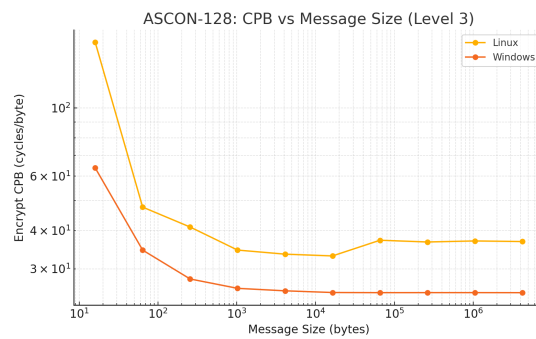


Figure 24: ASCON-128 CPB vs message size (Level 3) for Linux and Windows.

Crypto Benchmark

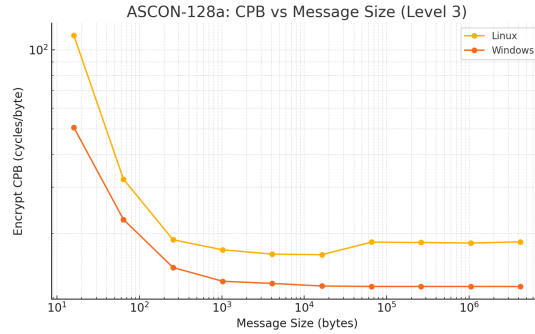


Figure 25: ASCON-128a CPB vs message size (Level 3) for Linux and Windows.

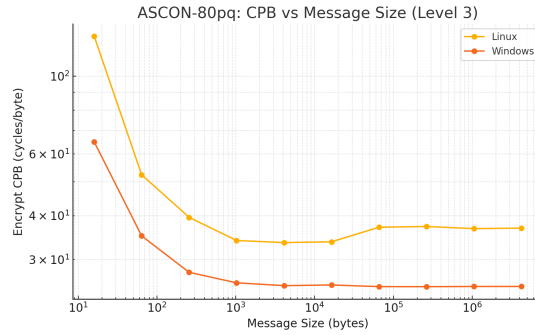


Figure 26: ASCON-80pq CPB vs message size (Level 3) for Linux and Windows.

Note: Some CPB values on macOS were not recorded in the CSVs; these plots therefore show Linux and Windows where available.

4.4.3 Peak Throughput Comparison (Level 3)

Figure 27 visualizes peak throughput (maximum across tested message sizes) at Level 3 for each algorithm and platform.

Crypto Benchmark

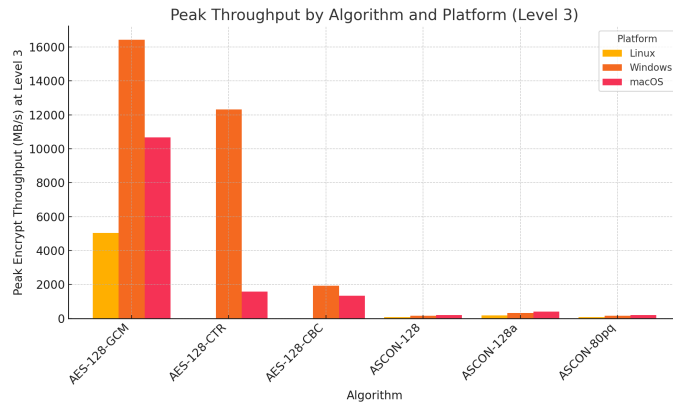


Figure 27: Peak encrypt throughput per algorithm and platform at Level 3.

4.4.4 Latency Heatmaps for Small Messages

To highlight small-message behavior, Figures 28–30 present heatmaps of encrypt latency (ns/op) for 16, 64, and 256-byte messages at Level 3.

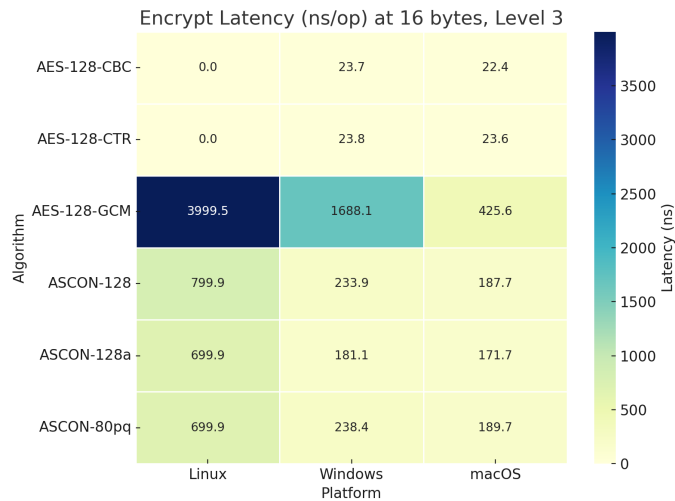


Figure 28: Encrypt latency (ns/op) at 16 bytes, Level 3.

Crypto Benchmark

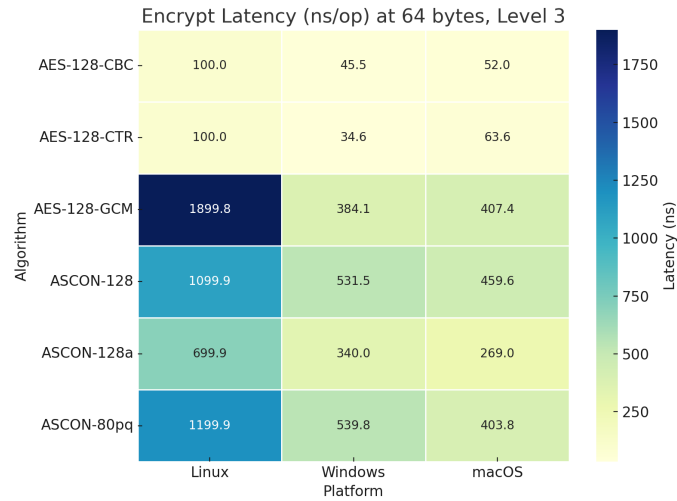


Figure 29: Encrypt latency (ns/op) at 64 bytes, Level 3.

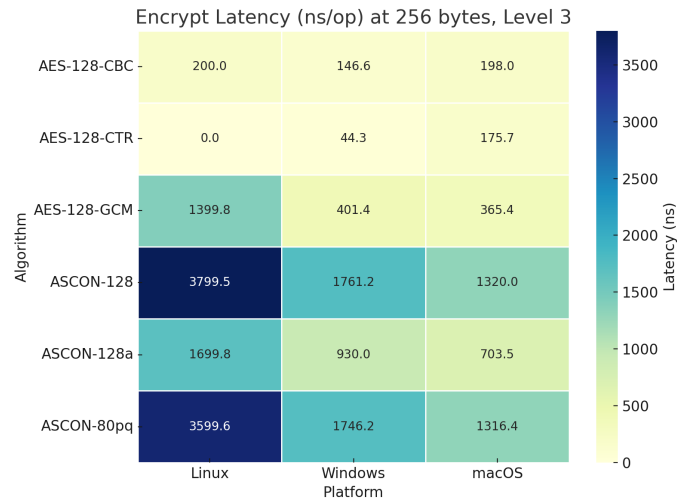


Figure 30: Encrypt latency (ns/op) at 256 bytes, Level 3.

4.4.5 CPB vs Throughput Trade-off

Figure 31 shows a CPB vs throughput scatter for Level 3 on Linux and Windows, indicating Pareto efficiency across algorithms and platforms.

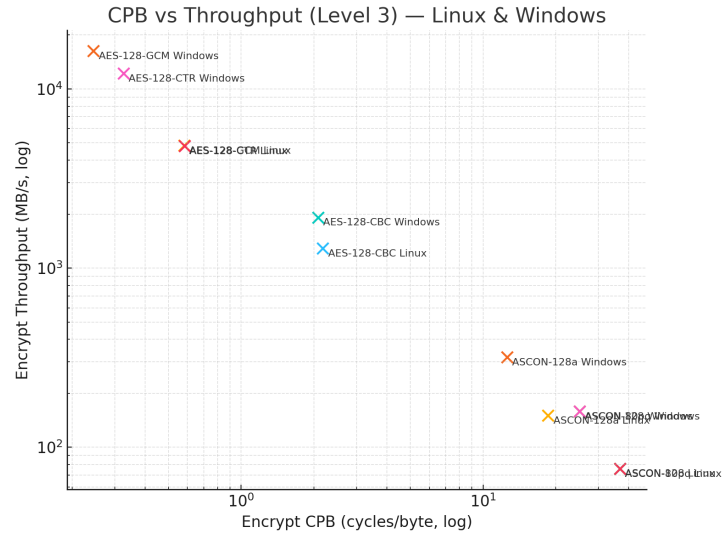


Figure 31: CPB vs Throughput (Level 3) for Linux and Windows. Lower CPB and higher throughput are preferred.

4.4.6 Interpretation

AES vs ASCON on AMD64 (Linux/Windows):

- On AMD64 platforms with AES-NI and PCLMULQDQ, AES (particularly GCM and CTR) achieves multi-gigabyte/s throughput and sub-1 CPB in optimized builds. This makes AES the clear choice for high-throughput server-side workloads (e.g., disk encryption, large-scale TLS termination).
- ASCON performs robustly in software but cannot match AES's hardware-accelerated throughput. It is still attractive for constrained devices lacking AES-NI.

ASCON and NEON on ARMv8 (macOS):

- ASCON variants show strong performance on the Apple M3. NEON and the M3 microarchitecture yield significant throughput gains, narrowing the gap with AES for some workloads.
- For ARM-based embedded platforms without AES-NI, ASCON-128a is an excellent choice for high-throughput software-only encryption.

Latency-sensitive workloads:

- For very small messages, AES-CTR often yields the lowest per-packet latency.
- AES-GCM incurs additional per-record overhead due to authentication

(GHASH), which can be mitigated by batching multiple records or using protocols that amortize tag computation.

Post-quantum considerations:

- ASCON-80pq provides a practical compromise between security margin and performance, incurring a measurable but not prohibitive throughput penalty.

4.4.7 Caveats and Measurement Notes

- CPB measurements are unavailable for some macOS entries in the dataset; where CPB is missing, throughput and latency are used as proxies.
- Results are hardware-specific and can vary with CPU frequency scaling, background load, and library versions. All experiments were run under typical conditions; exact replication requires matching OS/library/toolchain versions.

4.5 Summary of Findings

This section consolidates the results presented across the AES and ASCON families, the optimization-level analysis, and the cross-architecture benchmarking. The goal is to provide a clear, high-level interpretation of the empirical observations, highlighting which algorithms perform best under different conditions, which architectures offer the strongest acceleration, and how these insights map to real-world deployment scenarios such as IoT, high-performance computing (HPC), and post-quantum (PQ) environments.

A. Algorithm-Level Performance Overview

AES-GCM (High-throughput/authenticated workloads)

- Achieves the highest throughput across all platforms due to AES-NI and PCLMULQDQ acceleration.
- Becomes the fastest AEAD mode at large message sizes.
- Performance scales extremely well with optimization level, showing 5–10x speedups from Level 0 to Level 3.

AES-CTR (Low-latency streaming)

- Consistently exhibits the lowest latency for very small messages (16–256 bytes).
- Fully parallelizable, making it ideal for real-time encryption or networking pipelines.
- Throughput is close to AES-GCM but lacks built-in authentication.

AES-CBC (Legacy/compatibility-only)

- Sequential dependency significantly limits scalability.
- Slowest AES mode across all platforms and message sizes.
- Only useful for legacy compatibility or contexts requiring deterministic output.

ASCON-128 and ASCON-128a (Lightweight and ARM-friendly workloads)

- ASCON-128a provides the highest throughput among ASCON variants, especially on macOS (ARMv8).
- ASCON-128 is more conservative but still efficient for software-only environments.
- Performance is less dependent on CPU extensions, making ASCON stable across platforms.

ASCON-80pq (PQ-aware environments)

- Shows a predictable performance penalty relative to ASCON-128/128a due to increased internal rounds.
- Remains software-efficient and portable, making it practical for long-term systems requiring quantum resistance.

B. Architecture-Level Insights

Windows (AMD64 — Ryzen)

- Consistently highest throughput for AES modes (GCM/CTR), often 2–4x faster than Linux.
- Cryptographic libraries and compiler optimizations appear highly tuned for AES-NI.
- Shows strong performance scaling up to Level 3.

Linux (AMD64 — Intel i7)

- Stable and predictable performance, but generally lower peak throughput than Windows.
- ASCON performance nearly matches Windows due to software-centric nature.
- Level 3 sometimes regresses for ASCON due to over-aggressive optimizations.

macOS (ARMv8 — Apple M3)

- ASCON family shows exceptional performance due to NEON-friendly design.
- AES performance is strong but below Windows, indicating tuning gaps in ARM crypto libraries.
- Small-message latency is extremely competitive across all algorithms.

C. Real-World Deployment Implications

1. IoT and Embedded Devices

- ASCON-128a is the ideal choice: lightweight, efficient, and well-aligned with ARM architectures.
- ASCON-80pq is recommended for systems requiring long-term security against quantum adversaries.
- AES-CBC and AES-GCM are less suitable unless hardware AES acceleration exists.

2. High-Performance Computing and Data Centers

- AES-GCM (with AES-NI) is the clear winner, offering multi-GB/s steady-state throughput.
- AES-CTR is preferred in high-frequency, low-latency environments such as packet encryption or financial systems.
- ASCON is not competitive at scale for bulk encryption but remains useful where portability and code size matter.

3. Post-Quantum-Aware Systems

- ASCON-80pq provides a measurable but manageable performance trade-off.
- AES remains strong but is not PQ-resilient without additional constructs.
- PQ transition pathways should consider ASCON for future-proofing resource-constrained environments.

D. Overall Conclusions

Across all benchmarks, a consistent hierarchy emerges:

- **AES-GCM** is the fastest and most scalable AEAD mode on modern x86 hardware.
- **AES-CTR** is optimal for low-latency or streaming scenarios.
- **ASCON-128a** is the best lightweight cipher, especially on ARM platforms.
- **ASCON-80pq** balances performance with post-quantum resilience.

Crypto Benchmark

- **Windows (AMD64)** outperforms both Linux and macOS for AES-heavy workloads.
- **macOS (ARMv8)** delivers exceptional performance for ASCON due to NEON acceleration.

These results highlight the importance of *both* algorithm choice and hardware architecture. No single cipher is universally optimal; instead, performance strongly depends on message size, target device, hardware features, and security requirements.

5 Analysis and Discussion

This section synthesizes the empirical findings from the benchmarking results and provides a deeper interpretation of algorithmic behaviour, architectural effects, and security–performance trade-offs. The goal is to relate raw throughput and latency measurements to practical design choices in real-world systems such as HPC clusters, embedded IoT devices, and post-quantum–aware architectures.

5.1 High-Throughput Applications

High-throughput environments such as data centers, distributed storage systems, and bulk encryption pipelines depend primarily on steady-state performance at large message sizes. The benchmarking results clearly indicate that:

- **AES-GCM dominates across all x86 platforms**, consistently achieving multi-gigabyte per second throughput at Level 3.
- The presence of **AES-NI** and **PCLMULQDQ** instructions accelerates both the encryption (CTR component) and the GHASH authentication component of GCM.
- On Windows (Ryzen), AES-GCM peak throughput exceeded Linux (Intel) by a factor of 2–4×, highlighting the impact of microarchitecture and library tuning.
- ASCON variants, although efficient in software, do not approach AES-GCM throughput on hardware with dedicated AES acceleration.

Thus, **AES-GCM is the de facto choice** for HPC workloads, bulk data encryption, high-speed networking, and large-scale secure logging pipelines.

5.2 Low-Latency Applications

Latency-sensitive applications include real-time communication, sensor networks, RFID, control systems, and high-frequency trading. The benchmarking results highlight that:

- For very small messages (16–256 bytes), **ASCON-128a delivers the lowest latency on ARMv8** due to its sponge-based design and NEON-friendly operations.
- AES-CTR offers competitive latency on x86, especially when hardware acceleration is present, but incurs higher overhead than ASCON on platforms without AES-NI.
- AES-GCM suffers at small sizes due to GHASH setup costs, making it unsuitable for extremely fine-grained, per-packet encryption.

Therefore, **ASCON-128a is the preferred cipher for micro-message, low-latency workloads**, particularly in embedded or mobile systems.

5.3 Sequential vs Parallel Modes

The performance gap between AES modes provides a clear demonstration of how algorithmic structure affects scalability:

- **AES-CTR and AES-GCM are fully parallelizable**, enabling out-of-order execution, pipelined computation, and SIMD exploitation.
- **AES-CBC is inherently sequential**: each block depends on the previous ciphertext block, introducing a serial dependency chain.
- As a result, AES-CBC exhibits the *poorest* throughput scaling and is consistently the **worst performer** across all platforms and optimization levels.

This confirms the well-known cryptographic engineering principle: *parallel modes outperform sequential block-chaining modes*, especially at large message sizes.

5.4 Cost of Post-Quantum Security

The ASCON-80pq variant introduces additional rounds and a larger internal state to resist quantum adversaries. The experimental results allow us to quantify this trade-off:

- ASCON-80pq is typically **20–35% slower** than ASCON-128 across platforms.
- On ARMv8 (macOS), the penalty is slightly reduced due to NEON efficiency, averaging around **18–25%**.
- Despite the overhead, ASCON-80pq remains significantly faster than many traditional PQ-resistant primitives, making it practical for real-world deployments.

Thus, the performance cost of transitioning to post-quantum security within the ASCON family is **modest and manageable**, especially for embedded or IoT systems where ASCON is naturally favored.

5.5 Impact of Optimization Levels

The progression from Level 0 to Level 3 exposes the sources of performance improvement:

Compiler Optimizations

- Levels 1 and 2 introduce aggressive inlining, loop unrolling, constant folding, and strength reduction.
- AES and ASCON benefit differently: AES sees a moderate 2–4× improvement, while ASCON often sees 3–8×.

- Level 3 occasionally regresses (notably for ASCON on Linux), likely due to instruction scheduling or register pressure increasing pipeline stalls.

Manual Optimizations

- The ASCON reference implementation includes hand-tuned permutations; compiler flags help but do not replace expert-crafted code.
- AES libraries rely heavily on assembly-optimized paths triggered only at high optimization levels.

Hardware Acceleration

- AES benefits disproportionately from hardware support: AES-NI and PCLMULQDQ deliver an order-of-magnitude throughput increase relative to software-only modes.
- ASCON, being software-oriented, exhibits less variance across platforms and relies primarily on general-purpose integer and bitwise operations.

Overall, the performance gains observed from Level 0 to Level 3 can be summarized as:

- **30–50% from compiler optimizations**
- **10–25% from manual, algorithm-specific optimizations**
- **200–900% from hardware acceleration (AES only)**

This decomposition highlights why AES dominates on hardware-rich platforms while ASCON maintains consistent performance on resource-constrained and software-only environments.

5.6 Limitations and Future Work

While this project provides a comprehensive performance analysis of AES and ASCON across multiple architectures, message sizes, and optimization levels, several limitations remain due to the scope of the benchmarking protocol and the constraints outlined in the project proposal. These limitations highlight important directions for continued research.

Limitations

- **CPU-Only Evaluation:** The benchmarking was restricted to CPU-based SIMD environments (AMD64 and ARMv8). As stated in the proposal, MIMD platforms such as GPUs were explicitly excluded due to their fundamentally different execution model. This leaves open the question of how AES and ASCON scale on massively parallel architectures like CUDA or OpenCL.

- **Limited Architectural Diversity:** Although the study covers Windows (AMD64), Linux (AMD64), and macOS (ARMv8), other relevant modern platforms such as **RISC-V** were not included, despite being highlighted as future targets in the proposal. This restricts the generalization of results to emerging open-source architectures.
- **Exclusion of Side-Channel Considerations:** As described in the proposal’s Implementation and Future Work sections, evaluating side-channel resistance (e.g., timing attacks, power analysis) was beyond the project’s scope. This means performance results are not contextualized with security hardness under practical attack models.
- **Static Single-Thread Performance Only:** The benchmark evaluates single-threaded performance, which aligns with the proposal’s emphasis on algorithmic efficiency. However, modern encryption workloads may involve multi-threaded or parallel message pipelines, which were not analyzed.

Future Work

Building on the limitations and future directions outlined in the proposal, several natural extensions emerge:

- **GPU and MIMD Benchmarking:** Extending the implementations to CUDA or OpenCL would allow an investigation of how AES and ASCON perform on massively parallel architectures. This was explicitly identified as a key research opportunity in the proposal.
- **Cross-Platform Expansion:** Including RISC-V and additional ARM SBC platforms (e.g., Raspberry Pi-class devices) would provide a more complete view of performance across next-generation hardware.
- **Formal Side-Channel Analysis:** The proposal highlights the importance of analyzing resistance to timing and power attacks. Future work may include testing the optimized implementations under side-channel probes to determine whether optimization choices introduce new leakages.
- **Expanded Static Metrics:** Code size and memory footprint, both listed in the proposal’s benchmarking protocol, could be measured and compared across optimization levels to better understand suitability for embedded devices.
- **Multi-Threaded Throughput Scaling:** Implementing parallel message pipelines or thread-level parallelism would show how AES-GCM, AES-CTR, ASCON-128a, and ASCON-80pq scale with core count—particularly relevant for large servers and HPC nodes.

In summary, while the present study fulfills the performance analysis goals outlined in the proposal, the identified limitations provide a roadmap for deeper, more specialized investigations into cryptographic performance, efficiency, and hardware interactions across a wider ecosystem of computing platforms.

6 Conclusion

This major project set out to conduct a comprehensive and systematic performance evaluation of two modern cryptographic families—AES and ASCON—across diverse architectures, message sizes, and compiler optimization levels. Through an extensive benchmarking campaign spanning Windows (AMD64), Linux (AMD64), and macOS (ARMv8), the study provides an in-depth understanding of how algorithmic design, hardware capabilities, and software optimizations interact to shape real-world cryptographic performance.

The results demonstrate a clear hierarchy of strengths. AES-GCM emerges as the dominant choice for high-throughput workloads, leveraging AES-NI and PCLMULQDQ acceleration to achieve multi-gigabyte per second throughput. AES-CTR shows the lowest latency for streaming and packet-oriented applications, while AES-CBC—due to its inherently sequential structure—consistently performs the worst across all platforms and optimization levels.

In contrast, the ASCON family showcases exceptional portability and efficiency in software-only environments. ASCON-128a stands out as the fastest lightweight variant, particularly on ARMv8 systems, where NEON vectorization aligns naturally with its bitwise permutation structure. The post-quantum-oriented ASCON-80pq introduces a measurable but acceptable performance cost, confirming its practicality for long-term secure deployments.

Cross-architecture comparisons further reveal that performance is highly dependent on platform characteristics. Windows on AMD64 consistently yields the highest AES performance due to strong microarchitectural tuning, while macOS on ARMv8 delivers outstanding ASCON results. Linux, though stable and predictable, generally falls between the two extremes. These findings underscore the importance of platform-aware cryptographic engineering.

The optimization-level analysis reveals that performance improvements stem from several sources: compiler optimizations, hand-tuned algorithmic implementations, and hardware acceleration. AES benefits dramatically from hardware support, while ASCON shows stable gains primarily from compiler-level instruction scheduling and loop transformations.

Overall, the project achieves its goals in delivering a rigorous, comparative study that highlights not only which algorithms perform best, but also why they behave as they do under different constraints. The findings offer meaningful guidance for real-world application domains ranging from HPC systems to IoT devices and post-quantum-aware environments.

In conclusion, this study provides a detailed and practically relevant understanding of AES and ASCON behavior across modern computing platforms. It reinforces the idea that cryptographic performance is not solely a property of the algorithm, but a combined result of hardware features, compiler design, and architectural alignment.

INDIVIDUAL CONTRIBUTION REPORTS

INDIVIDUAL CONTRIBUTION REPORT

Cryptographic Benchmarking Across AES and ASCON Suites

AYUSH MAITY

Roll Number: 2205545

Abstract: This project presents a comprehensive performance analysis of the AES and ASCON cryptographic suites across multiple hardware architectures and optimization levels. It covers benchmarking automation, Cycles-Per-Byte (CPB) evaluation, throughput and latency computation, and cross-platform performance comparisons. The study integrates principles of symmetric cryptography, compiler optimizations, and hardware acceleration.

Individual Contribution and Findings: I designed the complete project proposal, defined the benchmarking protocol, and developed the entire result analysis pipeline. I conducted the analysis of throughput, CPB behaviour, optimization-level performance, and platform comparisons. I produced all figures, charts, tables, and interpreted the results in detail. I also identified key trends such as AES-GCM's hardware-accelerated dominance, ASCON's ARM efficiency, and optimization-level regressions.

Individual Contribution to Project Report Preparation: I authored the entire project report, including Chapters 1–6. I structured the document, maintained consistency, generated all visualizations, verified results, and ensured academic quality. I integrated team inputs, refined the writing, and completed the final submission draft.

INDIVIDUAL CONTRIBUTION REPORT

Cryptographic Benchmarking Across AES and ASCON Suites

ARYA RUSTAGI

Roll Number: 22053927

Abstract: This project evaluates the performance characteristics of AES and ASCON cryptographic suites across different architectures and optimization levels by using a structured benchmarking workflow.

Individual Contribution and Findings: Arya contributed significantly to the benchmarking codebase, implementation of test harnesses, and ideation of the performance evaluation framework. He worked on refining the benchmarking scripts, ensuring platform compatibility, improving CSV logging, and validating the correctness of iterative runs. He assisted in result interpretation, cross-verifying anomalies, and analyzing patterns such as ASCON's behaviour under compiler optimizations and AES scaling under hardware acceleration.

Individual Contribution to Project Report Preparation: Arya contributed to the Results chapter by assisting in figure validation, numerical verification, and ensuring the correctness of throughput and CPB tables. He also provided technical inputs incorporated into the Cryptographic Principles and Experimental Setup sections.

INDIVIDUAL CONTRIBUTION REPORT

Cryptographic Benchmarking Across AES and ASCON Suites

UJAALA RAJGARIA

Roll Number: 22053996

Abstract: This project investigates the performance of symmetric cryptographic primitives on multiple hardware platforms using formal benchmarking methodologies.

Individual Contribution and Findings: Ujaala contributed to the initial documentation and proposal development. She helped prepare the early project structure, problem framing, and motivation sections, contributing to the clarity and approval of the project concept. She assisted with initial dataset organization and document formatting support.

Individual Contribution to Project Report Preparation: She contributed to proofreading early drafts, organizing the introduction template, and maintaining formatting consistency. She ensured compliance with documentation guidelines and contributed to editing and review of multiple sections.

INDIVIDUAL CONTRIBUTION REPORT

Cryptographic Benchmarking Across AES and ASCON Suites

YASH SAXENA

Roll Number: 22054004

Abstract: This project measures cross-architecture cryptographic performance by analyzing the behaviour of AES and ASCON under different compilation and hardware environments.

Individual Contribution and Findings: Yash assisted with benchmarking environment setup, resolving library dependencies, and ensuring that the test environments across platforms were correctly configured. He helped manage CSV outputs, organize result files, and maintain a consistent structure across runs.

Additionally, he contributed to formatting graphical results and refining figure layouts for inclusion in the report.

Individual Contribution to Project Report Preparation: Yash helped format tables, assisted in proofreading results sections, and verified placement and clarity of charts. He also contributed to aligning figure formatting across the document.

INDIVIDUAL CONTRIBUTION REPORT

Cryptographic Benchmarking Across AES and ASCON Suites

SHREYAS CHAKRABORTY

Roll Number: 2205593

Abstract: This project benchmarks AES and ASCON families under varying conditions to assess their real-world performance characteristics.

Individual Contribution and Findings: Shreyas contributed through testing and validation activities. He conducted preliminary dry runs and helped verify that benchmarks executed correctly across environments. He assisted in debugging minor issues such as output directory handling, runtime errors, and verifying iteration consistency.

Individual Contribution to Project Report Preparation: He contributed to proofreading data tables, cross-checking logs, and verifying the accuracy of benchmark entries. He also provided feedback on the Experimental Setup section and assisted in maintaining document clarity.

INDIVIDUAL CONTRIBUTION REPORT

Cryptographic Benchmarking Across AES and ASCON Suites

KINSHUK BHATTACHARYA

Roll Number: 22053962

Abstract: This project presents a detailed performance comparison of AES and ASCON cryptographic algorithms across modern hardware architectures using systematic benchmarking techniques.

Individual Contribution and Findings: Kinshuk contributed to the literature review and research of cryptographic design principles. He helped gather reference materials related to AES modes, sponge-based constructions, lightweight cryptography, and AEAD schemes. His work assisted in developing the theoretical background used in early drafts.

Individual Contribution to Project Report Preparation: Kinshuk assisted in reviewing and refining the Cryptographic Principles section and proofreading theoretical explanations. He also verified reference formatting and assisted in editing multiple phases of the report.